Implementation Plan

TigerWatch Platform
Automated Camera Trap Triage &
Tiger Movement Intelligence System

Pench Tiger Reserve

Document Version:	1.0
Date:	August 16, 2026
Type:	Full-Stack Implementation Plan
Synthesized From:	PRD v1.0, TRD v1.0, UI/UX v1.0, Website Flow v1.0, Backend Schema v1.0
Timeline:	7 Phases / 14 Weeks
Backend:	Python 3.11+ / FastAPI / PostgreSQL 16 / Celery+Redis / MinIO
Frontend:	Next.js 14 / React 18 / Leaflet.js / D3.js / Vanilla CSS
AI/ML:	MegaDetector V6 / YOLOv8-L / Siamese CNN / SciPy KDE
Infrastructure:	Docker Compose / Nginx / Prometheus+Grafana+Loki

Table of Contents

11. Risk Register
12. Open Questions & Decisions Required

1. Project Scope & Source Documents

This implementation plan synthesizes specifications from five companion documents into a single actionable build guide. The TigerWatch platform is a full-stack AI/ML system for camera trap triage, individual tiger identification, spatial occupancy analysis, and behavioural deviation alerting.

1.1 Source Documents

Document	Pages	Key Content
PRD v1.0	27	Problem statement, functional requirements, success metrics, stakeholder needs
TRD v1.0	36	Technical algorithms, DDL schemas, API catalogue, performance SLAs
UI/UX Design System v1.0	31	Deep Wilderness theme, color tokens, component library, page designs
Website Flow Guide v1.0	30	Sitemap, routes, user journeys, component hierarchy, build sequence
Backend Schema Plan v1.0	~30	12 DB tables, 45 endpoints, pipeline architecture, deployment topology

1.2 Technology Stack

Layer	Technology	Purpose
Language	Python 3.11+	Backend, ML pipeline
API Framework	FastAPI + Uvicorn	REST API with OpenAPI 3.1 auto-docs
Database	PostgreSQL 16 + PostGIS 3.4	Relational + spatial data store
Vector Store	pgvector 0.5+	512-d embedding similarity search (IVFFlat)
Task Queue	Celery + Redis 7	Async ML inference & batch processing
Object Storage	MinIO	S3-compatible image storage with lifecycle policies
Frontend	Next.js 14 (App Router)	React 18 SPA with server-side features
Maps	Leaflet.js + dark tiles	Interactive reserve map with GeoJSON overlays
Charts	D3.js + Recharts	Data visualization (timelines, heatmaps, trends)
Blank Filtering	MegaDetector V6 (YOLOv9-C)	Camera trap triage via PyTorch-Wildlife
Tiger Detection	YOLOv8-L (fine-tuned)	Tiger-specific object detection
Re-Identification	Siamese CNN (DenseNet-169)	Stripe pattern embedding extraction
Spatial Analytics	SciPy, GeoPandas, R (rpy2)	KDE home-range, MCP, overlap analysis
Containerization	Docker + Docker Compose	All services containerized
Web Server	Nginx	TLS termination, reverse proxy, static assets
Monitoring	Prometheus + Grafana + Loki	Metrics, dashboards, log aggregation

2. Core Modules Summary

The system is organized into four core AI/ML modules, each implemented as a stage in the processing pipeline:

Module 1: Blank Image Filtering

- Model: MegaDetector V6 (YOLOv9-C architecture, 1280x1280 input resolution)
- Classification: BLANK / SUBJECT / BOUNDARY with configurable confidence threshold (default 0.20)
- Quarantine: safe, reversible deletion to MinIO quarantine bucket (30-day lifecycle)
- Burst logic: keeps top-1 image from 3-second windows even if borderline
- Performance: >=1,000 images/min on GPU, >=100 images/min on CPU fallback
- Success metric: >95% recall, <2% false negative rate

Module 2: Individual Tiger Identification

- Detection: Fine-tuned YOLOv8-L (min confidence 0.40) for tiger-specific detection
- Crop: bounding box + 15% padding, resized to 448x448, flank L/R classification
- Embedding: Siamese CNN (DenseNet-169 backbone, Triplet Loss) outputs L2-normalized 512-d vector
- Matching: cosine similarity via pgvector <=> operator, same-flank-only comparison
- Thresholds: >=0.85 auto-match | 0.60-0.85 review queue | <0.60 new individual enrollment
- Success metric: Rank-1 accuracy >90%, <5s per image

Module 3: Spatial Occupancy Analysis

- Home range: KDE 95% isopleth (full range) + KDE 50% isopleth (core activity area)
- Secondary: Minimum Convex Polygon (MCP) as complementary metric
- Centroid: weighted activity centroid from capture locations
- Overlap: pairwise Volume of Intersection (VI) and Bhattacharyya's Affinity (BA) indices
- Output: GeoJSON polygons stored in PostGIS, exportable as Shapefile/KML/PDF
- Regenerated from scratch on every processing run (not incremental)

Module 4: Deviation & Trend Alerting

- RANGE_SHIFT: centroid displacement exceeds threshold (core: 15 sq km, buffer: 5 km linear). Priority: Medium-High.
- NOVEL_STATION: capture at station not in historical profile. Priority: variable (>5km HIGH, 2-5km MED, <2km LOW).
- BUFFER_APPROACH: capture at buffer/village-adjacent station not in profile. Priority: ALWAYS HIGH. Triggers SMS.
- PROLONGED_ABSENCE: regular individual not detected in current run. Priority: escalating (1st LOW, 2nd MED, 3rd+ HIGH).
- Delivery: WebSocket push to dashboard, email to configured recipients, SMS for HIGH-priority BUFFER_APPROACH.

3. Phase 1: Infrastructure & Database (Week 1-2)

[CRITICAL]
This phase establishes all foundational services. Nothing else can proceed without it.

3.1 Docker Compose Stack

All services run in Docker containers orchestrated by Docker Compose:

Service	Internal Port	Exposed Port	Resources/Notes
Nginx	--	80, 443	TLS termination, reverse proxy to FastAPI
FastAPI	8000	--	REST API + WebSocket (4 Uvicorn workers)
Celery Worker	--	--	ML inference tasks (GPU-enabled container)
Celery Beat	--	--	Scheduled tasks (backup trigger, health checks)
PostgreSQL 16	5432	SSH tunnel only	16+ GB RAM, 16 GB shared_buffers, PostGIS+pgvector
Redis 7	6379	--	Cache + Celery broker + sessions + rate limiting
MinIO	9000/9001	--	S3-compatible object storage, erasure coding
Prometheus	9090	--	Metrics collection from all services
Grafana	3000	Via Nginx /monitoring	Dashboards, alerting rules

> GPU Support: NVIDIA Container Toolkit configured for Celery worker container (runtime: nvidia).

> Networking: all services on internal Docker bridge network. Only Nginx ports (80/443) exposed to host.

> Volumes: persistent Docker volumes for PostgreSQL data, MinIO storage, model weights, Redis AOF.

> Environment: .env file with all secrets (DB password, JWT secret, MinIO keys, SMTP credentials).

3.2 Database Schema (12 Tables)

PostgreSQL 16 with PostGIS 3.4 and pgvector 0.5+ extensions. Execute DDL from Backend Schema Plan Section 4.2-4.3:

Table	Purpose	Key Columns
individuals	Tiger identity catalogue	tiger_id, baseline_range (GEOMETRY), centroid, status
captures	Camera trap image records	individual_id FK, station_id FK, location, flank_side, confidence
stations	Camera trap station registry	station_code, location (GEOMETRY), zone
embeddings	512-d stripe pattern vectors	individual_id FK, flank_side, vector(512), is_reference
alerts	Behavioural deviation alerts	alert_type, individual_id FK, confidence, evidence JSONB
processing_runs	Pipeline execution records	status, total_images, blanks_removed, tigers_detected
users	System user accounts	role (5 roles), password_hash (bcrypt), is_active
overlap_pairs	Territorial overlap matrix	tiger_a_id FK, tiger_b_id FK, vi_index, ba_index
audit_log	Immutable audit trail	user_id, action, resource_type, before/after JSONB
models	AI model registry	model_name, version, weights_path, is_active, metrics JSONB
system_config	Runtime configuration	key PK, value JSONB, description
quarantine	Quarantined blank images	image_path, confidence, run_id, expires_at

3.3 Indexing Strategy

Index Type	Table.Column(s)	Purpose
B-Tree	captures.individual_id	Fast joins to individuals table
B-Tree	captures.station_id	Fast joins to stations table
B-Tree	captures.run_id	Fast joins to processing_runs
Composite	captures(individual_id, captured_at DESC)	Efficient capture history queries
Partial	captures WHERE review_status='pending'	Fast review queue retrieval
BRIN	captures.created_at	Time-range partition pruning on large tables
GiST	captures.location	Spatial queries (ST_Within, ST_DWithin)
GiST	stations.location	Station proximity searches
GiST	individuals.baseline_range	Spatial intersection tests
GiST	individuals.centroid	Centroid proximity queries
IVFFlat	embeddings.vector (cosine_ops)	ANN search, nlist=ceil(sqrt(N))
B-Tree	alerts.alert_type, alerts.status	Filter by type and status
B-Tree	individuals.tiger_id, individuals.status	Unique ID + status filtering

3.4 MinIO Bucket Setup

Bucket	Content	Lifecycle Policy
raw-images/	Original camera trap images	Indefinite retention
thumbnails/	256x256 WebP thumbnails	Generated on first access, cached
crops/	Tiger flank crops (448x448)	Indefinite retention
quarantine/	Quarantined blank images	30-day auto-expiry (configurable)
models/	AI model weight files (.pt, .onnx)	All versions retained
exports/	Generated GeoJSON, Shapefile, PDF	7-day auto-expiry

4. Phase 2: Backend API Layer (Week 2-4)

4.1 Application Structure

```
backend/
+-- app/
|   +-- main.py              # FastAPI app, CORS, middleware, lifespan
|   +-- config.py            # Settings (pydantic-settings, .env loading)
|   +-- database.py          # SQLAlchemy async engine + session factory
|   +-- models/              # SQLAlchemy ORM models (12 models)
|   +-- schemas/             # Pydantic request/response schemas
|   +-- routers/             # API route modules (9 routers)
|   |   +-- auth.py          # 3 endpoints (login, refresh, logout)
|   |   +-- users.py         # 5 endpoints (CRUD + disable)
|   |   +-- filtering.py     # 6 endpoints (run, status, report, restore, purge, config)
|   |   +-- identification.py # 8 endpoints (run, results, catalogue, images, review, merge)
|   |   +-- occupancy.py     # 6 endpoints (compute, individual, overlap, layers, export, summary)
|   |   +-- alerts.py        # 7 endpoints (generate, list, detail, update, trends, config, digest)
|   |   +-- stations.py      # 5 endpoints (CRUD + bulk CSV import)
|   |   +-- runs.py          # 5 endpoints (list, detail, compare, health, ready)
|   |   +-- dashboard.py     # Aggregated stats endpoint
|   +-- services/            # Business logic (processing, matching, alerting)
|   +-- middleware/           # Auth (JWT), RBAC, rate limiting, audit logging
|   +-- utils/               # Formatters, GPS helpers, file utilities
+-- tasks/                   # Celery task definitions
|   +-- pipeline.py          # Orchestration (chain stages)
|   +-- blank_filter.py      # Stage 1-2: ingest + MegaDetector
|   +-- tiger_detect.py      # Stage 3: YOLOv8-L detection + crop
|   +-- identity_match.py    # Stage 4: Siamese CNN + pgvector matching
|   +-- analytics.py         # Stage 5: KDE/MCP + alert generation
+-- migrations/              # Alembic migration files
+-- tests/                   # pytest test suite
```

4.2 API Endpoints (45 Total)

Group	Count	Key Endpoints
Authentication	3	POST /auth/login, POST /auth/refresh, POST /auth/logout
User Management	5	GET/POST/GET/PUT/DELETE /users
Filtering (Module 1)	6	POST /filtering/run, GET status, GET report, POST restore, DELETE purge
Identification (Module 2)	8	POST /identification/run, GET catalogue, GET review/queue, PUT review
Occupancy (Module 3)	6	POST /occupancy/compute, GET individual, GET overlap, GET export
Alerts (Module 4)	7	POST /alerts/generate, GET list, PUT update, POST config, GET digest
Stations	5	GET/POST/GET/PUT/DELETE /stations (+ bulk CSV import)
Runs & Health	5	GET /runs, GET /runs/{id}, GET compare, GET /health, GET /ready
Dashboard	~5	GET /dashboard/stats (aggregated KPIs, recent activity)

4.3 Authentication & RBAC

> JWT tokens: access token (1hr expiry, httpOnly cookie), refresh token (7d expiry, httpOnly cookie).

> Password storage: bcrypt with work factor 12, minimum 12 characters.

> 5-role RBAC with endpoint-level permission middleware:

Role	Dashboard	Catalogue	Upload	Review	Alerts	Map	Settings	GPS
ADMIN	Full	R/W	Yes	Yes	R/W	Yes	Yes	Full
BIOLOGIST	Full	R/W	Yes	Yes	R/W	Yes	Partial	Full
RANGE_OFFICER	Full	Read	Yes	--	R/W	Rounded	--	--
FIELD_STAFF	Limited	--	Yes	--	View	--	--	--
VIEWER	Full	Read	--	--	View	View	--	--

> Rate limiting: Redis sliding window - 100 req/min (authenticated), 20 (unauthenticated), 5 (heavy ops).

> Account lockout: 5 failures in 15 min -> 30-minute lockout with brute-force prevention.

4.4 WebSocket Channels

> ws/runs/{runId} - Live processing progress updates (every 100 images processed).

> ws/alerts - Real-time new alert push notifications to connected clients.

> ws/system - GPU utilization, memory, disk health status for admin monitoring.

> Implementation: PostgreSQL LISTEN/NOTIFY triggers WebSocket broadcast via FastAPI WebSocket endpoints.

5. Phase 3: AI/ML Processing Pipeline (Week 3-6)

[GPU REQUIRED]
NVIDIA RTX 3090/4090 (16+ GB VRAM) required. CPU fallback for Stage 1-2 only at reduced throughput.

The system processes camera trap data through a 5-stage sequential pipeline managed by Celery task queues:

```
STAGE 1        STAGE 2         STAGE 3          STAGE 4         STAGE 5
INGESTION  --> BLANK FILTER --> TIGER DETECT --> IDENTITY    --> ANALYTICS
                               & CROP           MATCHING       & ALERTING
* Upload       * MDv6 YOLOv9   * YOLOv8-L       * Siamese CNN   * KDE/MCP
* EXIF parse   * Threshold     * 15% pad crop   * Cosine sim    * Overlap
* Dedup        * Quarantine    * Quality QA     * Auto/Review   * Deviation
* Station      * Burst logic   * Flank L/R      * Enroll        * Alert gen
  assign                                                       * SMS/Email
```

5.1 Stage 1 - Ingestion

> Input modes: recursive directory scan (SD card) or ZIP upload (max 10GB via dashboard).
> EXIF parsing: DateTimeOriginal, GPS coords, Make/Model. Fallback: file mtime (flagged 'estimated').
> Station assignment: nearest station within 100m of EXIF GPS, or directory name regex, or CSV mapping.
> Deduplication: pHash 64-bit. Hamming distance 0 = exact duplicate (skip). <=5 = near-duplicate (flag, ingest).
> Cross-run dedup: check against last 3 runs for duplicates.
> Storage: originals uploaded to MinIO raw-images/ bucket with structured key: {run_id}/{station_code}/{filename}.
> Data standard: Camtrap DP format for interoperability with Wildlife Insights & LILA BC.

5.2 Stage 2 - Blank Filtering (MegaDetector V6)

> Model: YOLOv9-C architecture, 1280x1280 input resolution, FP16 mixed precision.
> Classification output: BLANK (no subject), SUBJECT (animal/person/vehicle detected), BOUNDARY (low-confidence edge case).
> Threshold: configurable (default 0.20). Scores < threshold -> quarantine. Burst logic: keep top-1 from 3s window.
> Quarantine: images moved to MinIO quarantine/ bucket. Database record with confidence score, restorable via API.
> Performance target: >=1,000 images/min (GPU batch size 32), >=100 images/min (CPU).

5.3 Stage 3 - Tiger Detection & Crop (YOLOv8-L)

> Model: fine-tuned YOLOv8-L on tiger-specific dataset, minimum confidence 0.40.
> Detection classes: animal, person, vehicle, tiger (only tiger detections proceed to Stage 4).
> Crop: bounding box + 15% padding, resized to 448x448 px. Stored in MinIO crops/ bucket.
> Flank classification: automatic L/R side determination from detection geometry and pose estimation.
> Quality scoring: blur detection (Laplacian variance), resolution check, occlusion assessment (0.0-1.0

score).

> Performance target: >=500 images/min on GPU.

5.4 Stage 4 - Identity Matching (Siamese CNN)

> Model: DenseNet-169 backbone, trained with Triplet Loss (margin=0.5), outputs L2-normalized 512-d vector.

> Matching: cosine similarity via pgvector <=> operator. Same-flank-only comparison (L vs L, R vs R).

> Threshold actions:

>> **>= 0.85 AUTO_MATCH:** Automatic assignment to existing individual. Reference embedding updated with 10% weight moving average.

>> **0.60 - 0.85 REVIEW_QUEUE:** Surfaced to human reviewer with top-5 candidates, side-by-side flank comparison, opacity slider overlay.

>> **< 0.60 NEW_INDIVIDUAL:** New enrollment: auto-generate ID (PTR-T-{NNN}), status=provisional, first embedding becomes reference.

> Performance target: <5s per image (including embedding extraction + vector search).

5.5 Stage 5 - Analytics & Alerting

Spatial Occupancy

> KDE: SciPy gaussian_kde (or R adehabitatHR via rpy2) computes 95% isopleth (full range) and 50% isopleth (core area).
> MCP: Minimum Convex Polygon as secondary range metric. Centroid: weighted activity centroid from all capture locations.
> Overlap: pairwise Volume of Intersection (VI) and Bhattacharyya's Affinity (BA) indices for all active tiger pairs.
> Storage: GeoJSON polygons stored in PostGIS GEOMETRY columns. Previous baselines overwritten each run.
> Performance: <30 seconds per individual, <30 minutes for full reserve (80 individuals).

Alert Generation

Alert Type	Trigger	Threshold	Priority
RANGE_SHIFT	Centroid displacement exceeds threshold	Core: 15 sq km. Buffer: 5 km	Med-High
NOVEL_STATION	Capture at station not in historical profile	>5km: HIGH. 2-5km: MED. <2km: LOW	Variable
BUFFER_APPROACH	Capture at buffer/village station, not in profile	Always triggered	ALWAYS HIGH
PROLONGED_ABSENCE	Regular individual not detected in current visit	1st: LOW. 2nd: MED. 3rd+: HIGH	Escalating

> Notification dispatch: WebSocket push (instant), email digest (configurable), SMS for HIGH BUFFER_APPROACH.
> Alert evidence: JSONB containing supporting image paths, GPS coordinates, distance measurements, bearing direction.

5.6 Pipeline Orchestration

> Celery chain: ingest.s() | filter.s() | detect.s() | match.s() | analyze.s() with error_callback for failure handling.
> Checkpointing: last_checkpoint field on processing_runs records last processed image ID. Resumable on failure.
> Progress reporting: WebSocket updates every 100 images. Dashboard shows live counters and ETA.
> Concurrent runs: only 1 active run at a time. Additional requests queued with user notification.

6. Phase 4: Frontend Foundation (Week 4-7)

6.1 Project Setup & Design Tokens

> Framework: npx create-next-app@latest ./ --typescript --app --no-tailwind --no-eslint
> Dependencies: lucide-react, swr, leaflet, react-leaflet, d3, recharts
> CSS approach: Vanilla CSS with CSS custom properties (BEM naming convention). No Tailwind.
> Design theme: 'Deep Wilderness' - dark mode first, warm amber (#B8822E) accent, forest green (#1A4D2E) secondary.
> Typography: Inter (400/500/600) + JetBrains Mono (400) from Google Fonts, 10-level type scale.

> Motion: expo-out easing cubic-bezier(0.16,1,0.3,1), 200-300ms transitions, prefers-reduced-motion support.

```
:root {
  --bg-deep: #030304; --bg-base: #060607; --bg-elevated: #0C0C0E;
  --bg-card: #101014; --surface: rgba(255,255,255,0.04);
  --fg-primary: #EDEDEF; --fg-secondary: #9CA3AF;
  --accent: #B8822E; --accent-bright: #D4993A;
  --accent-glow: rgba(184,130,46,0.25);
  --forest: #1A4D2E; --border: rgba(255,255,255,0.06);
  --radius-sm: 8px; --radius-lg: 16px;
  --ease-expo-out: cubic-bezier(0.16, 1, 0.3, 1);
  --font-sans: 'Inter', system-ui, sans-serif;
  --font-mono: 'JetBrains Mono', monospace;
}
```

6.2 Component Build Order (Atomic Design)

Build bottom-up: atoms -> molecules -> organisms -> templates -> pages:

Atoms (11 components - build first)

> Button (5 variants: primary/secondary/ghost/danger/icon), Input (text/search/number/password), Select, Badge, StatusDot (with pulse animation), Slider, Toggle, Tooltip, Spinner, Avatar, Icon (Lucide wrapper).

Molecules (9 components - build second)

> StatCard (label + value + trend + sparkline), TigerCard (thumbnail + ID + name + status), AlertCard (type bar + icon + title + confidence), CaptureRow, NavItem, FilterBar, BreadcrumbTrail, EmptyState, Toast.

Organisms (12 components - build third)

> Sidebar (logo + nav items + user section), TopBar (hamburger + breadcrumb + search + bell + user), CommandPalette (Cmd+K search modal), DataTable (sortable + paginated), TigerGrid, AlertFeed, ReviewPanel, MapView (Leaflet + layers + tiger selector), KPIRow, ProcessingPipeline (stepper), Modal, ImageLightbox.

Templates (3 layouts - build fourth)

> Layout A: Sidebar (260px) + TopBar (56px) + scrollable content. Used by: Dashboard, Catalogue, Alerts, Stations, Settings.
> Layout B: Sidebar + full-height Map + docked right panel (360px, collapsible). Used by: Occupancy Map.
> Layout C: No sidebar, content centered (max-width 800px). Used by: Image Review Queue.

7. Phase 5: Frontend Pages (Week 6-10)

7.1 Page Build Sequence (15 Pages)

Build in this exact order. Each page should be functional (with mock data if needed) before proceeding:

Step	Page	Route	Layout	Priority
1	Login	/login	Full-screen	CRITICAL
2	Layout Shell	(authenticated)/layout	Shell	CRITICAL
3	Dashboard	/dashboard	A	CRITICAL
4	Upload & Config	/processing/upload	A	CRITICAL
5	Run Monitor	/processing/runs/[runId]	A	CRITICAL
6	Tiger Catalogue	/catalogue	A	High
7	Tiger Profile	/catalogue/[tigerId]	A + Tabs	High
8	Occupancy Map	/map	B	High
9	Alert Feed	/alerts	A	High
10	Alert Detail	/alerts/[alertId]	A	High
11	Review Queue	/review	C	High
12	Station Management	/stations	A	Medium
13	Settings	/settings/*	A + Sub-nav	Medium

14	Command Palette	Global (Cmd+K)	Modal	Medium
15	Ambient Animations	Global background	Background	Low

7.2 Key Page Specifications

Dashboard (/dashboard)

- Welcome bar: 'Welcome back, [Name]' + 'Start New Run' primary CTA button.
- KPI cards (4-col bento grid): Total Tigers (trend), Images This Month (sparkline), Active Alerts (red if >0), Stations Online (progress ring).
- Two-column: population trend line chart (D3.js, 12 months) + activity feed (last 5 events, clickable).
- Two-column: mini Leaflet map (all tiger centroids as color dots, clickable) + alert donut chart by type.
- Recent runs table: last 5 runs with Run ID, date, status badge, image count, tigers found, alerts. Click row navigates.
- Auto-refresh every 60 seconds. Manual refresh button. Loading skeleton during refresh.

Tiger Profile (/catalogue/[tigerId])

- Header: tiger photo (200px) + ID (font-mono, accent) + name (H1, gradient) + status badge + sex badge + dates.
- 4 tabs: Overview (stats row + mini map + recent captures + overlap list), Captures (paginated table + lightbox), Map (full Leaflet with KDE + markers), Alerts (chronological alert list).
- Tab selection stored in URL query param (?tab=captures) for shareable deep links.

Occupancy Map (/map)

- Full-height Leaflet with dark tiles. Reserve boundaries (core=dashed green, buffer=dashed amber).
- Tiger selector in sidebar: checkboxes for each individual with color dots. Filter by ID/name.
- Layer controls (top-right): stations, KDE 95%, KDE 50%, centroids, overlap zones, buffer boundary, villages.
- Right panel (360px, docked): selected tiger details, range stats, overlap matrix heatmap, export buttons.
- Time slider (bottom): scrub through historical occupancy snapshots by processing run date.

Review Queue (/review)

- Full-width focus layout. Queue progress bar: 'X images pending review' (reviewed / total).
- Left (55%): query image (large flank crop) + metadata (station, date, flank, quality).
- Right (45%): top-5 candidate matches ranked by similarity (thumbnail + ID + name + score + date).
- Side-by-side comparison: click candidate for flank overlay with opacity slider.
- Actions: Confirm Match (primary) | New Individual (outlined) | Skip (ghost). Keyboard: 1-5, Enter, N, S.

8. Phase 6: Integration & Polish (Week 9-12)

8.1 API Integration

> SWR (stale-while-revalidate): all GET requests cached with SWR hooks. Auto-revalidate on window focus.
> Mutations: optimistic UI update -> API call -> on success: invalidate SWR cache + toast. On failure: rollback + error toast.
> WebSocket client: single connection on authenticated shell mount, multiplexed channels (processing, alerts, system).
> Token interceptor: on 401 response, auto-call POST /auth/refresh. If refresh fails, redirect to /login.

8.2 State Management

> AuthContext: user object, JWT tokens, role, login/logout actions. Persisted in httpOnly cookies.
> UIContext: sidebar collapsed state, active theme, mobile menu open. Persisted in localStorage.
> Server state: SWR with cache keys matching API URLs. Manual invalidation after mutations.

8.3 Responsive & Mobile

> Mobile (<768px): sidebar hidden, hamburger menu with full-screen overlay, single column, bottom tab bar for primary nav.
> Tablet (768-1023px): collapsible sidebar (icon-only 60px), 2-column grids, map full-width with overlay panels.
> Desktop (1024px+): full sidebar (260px), multi-column layouts, map with docked side panels.
> Touch: minimum 44x44px targets, swipe-left on alerts to acknowledge, pinch-to-zoom maps, pull-to-refresh.

8.4 Error & Empty States

> 7 error patterns: 400 (field-level), 401 (silent refresh), 403 (toast), 404 (full-page), 500 (full-page + retry), network (banner), timeout (toast + retry).
> 7 designed empty states: first-use dashboard, no tigers, filtered no-results, all-clear alerts, empty queue, no runs, no stations.
> Each empty state: themed SVG illustration + descriptive message + action button (e.g., 'Upload Images', 'Clear Filters').

8.5 Ambient Animations

> Floating gradient blobs: amber (#B8822E at 15% opacity, 900x1400px), forest (#1A4D2E at 12%, 600x800px), deep blue (#1E3A5F at 10%, 500x700px). Float animation 8-12s.
> Card hover: translateY(-4px), border brightens, shadow gains amber glow. 250ms expo-out.
> Entrance animations: staggered fade-up (opacity 0->1, y 24px->0, 80ms stagger per element).
> Map polygons: stroke-dasharray drawing animation, 800ms per polygon.
> prefers-reduced-motion: disables all animations, blobs become static gradients.

9. Phase 7: Testing & Deployment (Week 11-14)

9.1 Testing Strategy

Layer	Tool	Target
Backend Unit	pytest	80%+ coverage, all services and utilities
Backend Integration	pytest + testcontainers	Full pipeline stages with real PostgreSQL + Redis
API Contract	OpenAPI + Postman	All 45 endpoints validated against schema
Frontend Unit	React Testing Library	Component rendering, props, state
Frontend E2E	Cypress	Full user journeys (login -> upload -> review -> map -> alert)
ML Accuracy	Custom benchmark	Rank-1 accuracy, mAP, false negative rate on held-out set
Performance	k6 / locust	API p95 <200ms, dashboard load <3s, pipeline throughput targets

9.2 Docker Deployment

> Production docker-compose.yml with all 9 services (Nginx, FastAPI, Celery worker, Celery beat, PostgreSQL, Redis, MinIO, Prometheus, Grafana).
> Nginx: TLS termination (Let's Encrypt or self-signed for on-premise). Reverse proxy to FastAPI on port 8000.
> PostgreSQL tuning: shared_buffers=16GB, effective_cache_size=48GB, work_mem=256MB, WAL archiving enabled.
> GPU worker: NVIDIA Container Toolkit, FP16 mixed precision, model warmup (5 dummy images on startup).
> Health checks: Docker HEALTHCHECK on all services. Automatic restart policy: unless-stopped.

9.3 Monitoring & Observability

Category	Metrics	Tool
Application	Request count/latency per endpoint, Celery queue depth, processing progress	Prometheus
ML Models	Inference latency per model, GPU utilization, VRAM usage, blank ratio drift	Prometheus
Infrastructure	CPU/memory/disk, PostgreSQL connections, Redis hit rate, MinIO storage	Prometheus
Business	Catalogue size, new enrollments/month, alert count by type, review backlog	Grafana
Logging	Structured JSON logs with trace_id, 90d retention, 1y archive	Loki
System Alerts	GPU OOM, disk >85%, PG connections exhausted, API error >5%	Grafana

9.4 Backup & Disaster Recovery

Component	Method	Frequency	Retention	RPO
Database	pg_dump + WAL archiving (zstd + GPG)	Daily 2 AM IST	30d daily, 1y monthly	24 hrs
Images	MinIO mc mirror to external NAS/USB	Weekly + post-run	90 days	7 days
Configuration	Git-versioned Docker/env/certs	On every change	Indefinite	0 hrs
Model Weights	MinIO /models/ bucket	On upload	All versions	0 hrs

> RTO (same hardware): < 4 hours. RTO (replacement hardware): < 8 hours (including OS + Docker setup).
> Monthly DR drill: restore from backup on isolated VM, run full integration test suite, validate row counts.

10. Timeline & Gantt Overview

14-week implementation timeline with 7 phases. Phases 2-3 and 4-5 have overlap where frontend development proceeds in parallel with backend/ML work:

```
Week:  1   2   3   4   5   6   7   8   9  10  11  12  13  14
       |---|---|---|---|---|---|---|---|---|---|---|---|---|---|
P1: Infrastructure & Database
       [=======]
P2: Backend API Layer
           [===========]
P3: AI/ML Pipeline
               [===============]
P4: Frontend Foundation
                   [===============]
P5: Frontend Pages
                           [==================]
P6: Integration & Polish
                                       [===============]
P7: Testing & Deployment
                                               [===============]

MILESTONES:
  W2:  Database + Docker stack operational
  W4:  All 45 API endpoints functional
  W6:  Full ML pipeline tested end-to-end
  W7:  All frontend components built
  W10: All 15 pages functional with real data
  W12: Full integration tested, responsive + polished
  W14: Production deployment + monitoring live
```

Team Allocation (Recommended)

> Backend Engineer (1): Phases 1, 2, 4 (API layer), 6 (integration), 7 (deployment).

> ML Engineer (1): Phase 3 (all pipeline stages), model training/fine-tuning, accuracy benchmarking.

> Frontend Engineer (1): Phases 4, 5, 6, 8 (responsive/animations).

> DevOps (part-time): Phase 1 (Docker), Phase 7 (monitoring, backup, deployment).

> Solo developer path: sequential execution, ~18-20 weeks total. Start with mock data frontend.

11. Risk Register

Risk	Impact	Likelihood	Mitigation
No GPU hardware available	High	Medium	CPU fallback for Stage 1-2. Cloud GPU for Stage 3-4. Defer Stage 3-5.
Pre-trained tiger model unavailable	High	Medium	Use public MegaDetector V6 + general Re-ID. Fine-tune on Pench data later.
Pench GeoJSON boundaries unavailable	Medium	Low	Source approximate boundaries from OpenStreetMap. Replace when official data
Low connectivity at PTR headquarters	Medium	High	On-premise deployment. Offline tile cache. SMS alerts via local SIM.
Small tiger population (<20)	Low	Medium	IVFFlat index unnecessary. Use exact vector scan. System still functional.
Image quality varies (night, blur)	Medium	High	Quality scoring in Stage 3. Low-quality crops flagged for review, not auto-matche

Model accuracy drift over time	Medium	Medium	Monthly accuracy monitoring via Grafana. Retrain trigger on >5% accuracy drop.
Single server failure	High	Low	Daily backups + WAL archiving. RTO <4 hours. Monthly DR drills.

12. Open Questions & Decisions Required

[ACTION REQUIRED]

The following questions need your input before implementation begins.

>> **Q1: Build scope:** Do you want to build the full-stack application (backend + ML + frontend)? Or start with a specific layer? Recommendation: start backend + frontend in parallel with mock data, add ML pipeline when GPU hardware is available.

>> **Q2: Mock data vs real pipeline:** Should the frontend be built first with mock/seed data to demonstrate the UI, or should we implement the actual ML pipeline first (requiring GPU hardware)?

>> **Q3: Deployment target:** On-premise server at PTR headquarters (as specified in the Backend Schema Plan), or cloud-hosted for development/demo purposes (AWS/GCP/Azure)?

>> **Q4: Starting phase:** Which phase should we begin with? Recommended order: Phase 1 (Docker + Database) -> Phase 4 (Frontend Foundation) in parallel with Phase 2 (API Layer).

>> **Q5: ML model weights:** Do you have pre-trained weights for: (a) MegaDetector V6, (b) fine-tuned YOLOv8-L for tigers, (c) Siamese CNN for stripe re-identification? Or should we use publicly available pre-trained models and fine-tune later?

>> **Q6: Pench Reserve GeoJSON:** Do you have the core/buffer zone boundary GeoJSON files from the forest department? Or should we source approximate boundaries from OpenStreetMap for initial development?

>> **Q7: Team size:** Will this be built by a solo developer (sequential, ~18-20 weeks) or a team (parallel, ~14 weeks as planned)?

>> **Q8: Demo priority:** Is there a deadline for a working demo? If so, which features are must-have for the demo: (a) dashboard with mock data, (b) working blank filter, (c) full pipeline, (d) interactive map?

--- End of Document ---

Implementation Plan v1.0

TigerWatch - Pench Tiger Reserve Intelligence Platform

---

7 Phases | 14 Weeks | 12 Tables | 45 Endpoints | 5 Pipeline Stages | 15 Pages | 40+ Components